Image Edge Detection using Gradient (Vector Calculus Approach)

4th Semester – Vector Calculus Project

Name : Tilala Yash

Enrollment No: 2501442138

Roll no : 242328

Course : B.Tech – CSE (D2D)

Subject: Vector Calculus

Faculty Name: Ashish Sir

Topic 4:
Image Edge Detection
using Gradient (Vector Calculus)

Subject	Vector Calculus (4th Semester)
Topic	Image Edge Detection using Gradient
Language	Python (NumPy, SciPy, Matplotlib)
Key Concepts	Gradient, Partial Derivatives, Vector Fields, Magnitude, Direction
Operators	Sobel Operator, Prewitt Operator

1. Problem Statement

Edge detection is one of the most fundamental operations in image processing and computer vision. An edge in an image corresponds to a sharp change in pixel intensity — which, mathematically, is the region where the gradient of the image function is large.

The goal of this project is to implement image edge detection using vector calculus — specifically by computing the gradient of a 2D image function I(x, y). We demonstrate:

- How partial derivatives $\partial I/\partial x$ and $\partial I/\partial y$ represent intensity changes in horizontal and vertical directions.
- How the gradient vector $\nabla I = (\partial I/\partial x)\,\hat{\imath} + (\partial I/\partial y)\,\hat{\jmath}$ points in the direction of maximum intensity change.
- How the gradient magnitude $|\nabla I|$ identifies edge locations, and the direction $\theta = \arctan(G_y/G_x)$ gives edge orientation.
- Comparison of Sobel and Prewitt operators as practical discrete approximations of the gradient.

2. Mathematical Model

2.1 Image as a Scalar Field

An image is treated as a discrete 2D scalar field:

$$I : \mathbb{R}^2 \to \mathbb{R} \quad \text{where} \quad I = I(x, y) \quad \text{and} \quad I(x, y) \in [0, 255]$$

Each pixel at position (x, y) holds an intensity value. Brighter pixels have higher values; darker pixels have lower values.

2.2 Gradient of the Image (Vector Field)

The gradient of the scalar field I(x, y) is a vector field defined as:

$$\nabla I = (\partial I/\partial x)\ \hat{\imath} + (\partial I/\partial y)\ \hat{\jmath}$$

where:

- $\partial I/\partial x$ is the partial derivative with respect to x → detects vertical edges
- $\partial I/\partial y$ is the partial derivative with respect to y → detects horizontal edges
- $\hat{\imath}, \hat{\jmath}$ are the unit vectors in the x and y directions

2.3 Edge Magnitude and Direction

The magnitude of the gradient gives the edge strength at each pixel:

$$|\nabla I| = \sqrt{(\partial I/\partial x)^2 + (\partial I/\partial y)^2}$$

The direction of the gradient (perpendicular to the edge) is:

$$\theta = \arctan\left((\partial I/\partial y) \,/\, (\partial I/\partial x) \right)$$

2.4 Sobel Operator (Discrete Gradient Approximation)

In a discrete image, derivatives are approximated by convolution with Sobel kernels:

```
Kx = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]]   (x-gradient)
Ky = [[-1,-2,-1], [ 0, 0, 0], [ 1, 2, 1]]   (y-gradient)
```

$Gx = I * Kx$ (convolution)

$Gy = I * Ky$ (convolution)

$|\nabla I| = \sqrt{(Gx^2 + Gy^2)}$

3. Algorithm

- Step 1: Generate / load a grayscale image $I(x, y)$
- Step 2: Define Sobel kernels Kx and Ky
- Step 3: Convolve the image with Kx to get $Gx = \partial I/\partial x$
- Step 4: Convolve the image with Ky to get $Gy = \partial I/\partial y$
- Step 5: Compute gradient magnitude $|\nabla I| = \sqrt{(Gx^2 + Gy^2)}$
- Step 6: Compute gradient direction $\theta = \arctan(Gy / Gx)$
- Step 7: Apply double threshold to classify strong and weak edges
- Step 8: Visualise — edge map, vector field, magnitude heatmap, direction map

4. Code Implementation

4.1 Dependencies

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.ndimage import convolve
from PIL import Image
```

4.2 Sobel Gradient Function

```
def sobel_gradients(img):
    Kx = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=np.float64)
    Ky = np.array([[-1,-2,-1], [ 0, 0, 0], [ 1, 2, 1]], dtype=np.float64)
    Gx = convolve(img, Kx)          # ∂I/∂x
    Gy = convolve(img, Ky)          # ∂I/∂y
    magnitude = np.sqrt(Gx**2 + Gy**2)   # |∇I|
    direction = np.arctan2(Gy, Gx)        # θ
    return Gx, Gy, magnitude, direction
```

4.3 Double Threshold Edge Classification

```
def apply_threshold(magnitude, low=30, high=80):
    strong = magnitude >= high
    weak   = (magnitude >= low) & (magnitude < high)
    result = np.zeros_like(magnitude)
    result[strong] = 255    # definite edges
    result[weak]   = 128    # candidate edges
    return result
```

5. Graphs and Visualisations

5.1 Main Comparison: Original → Gradients → Edges

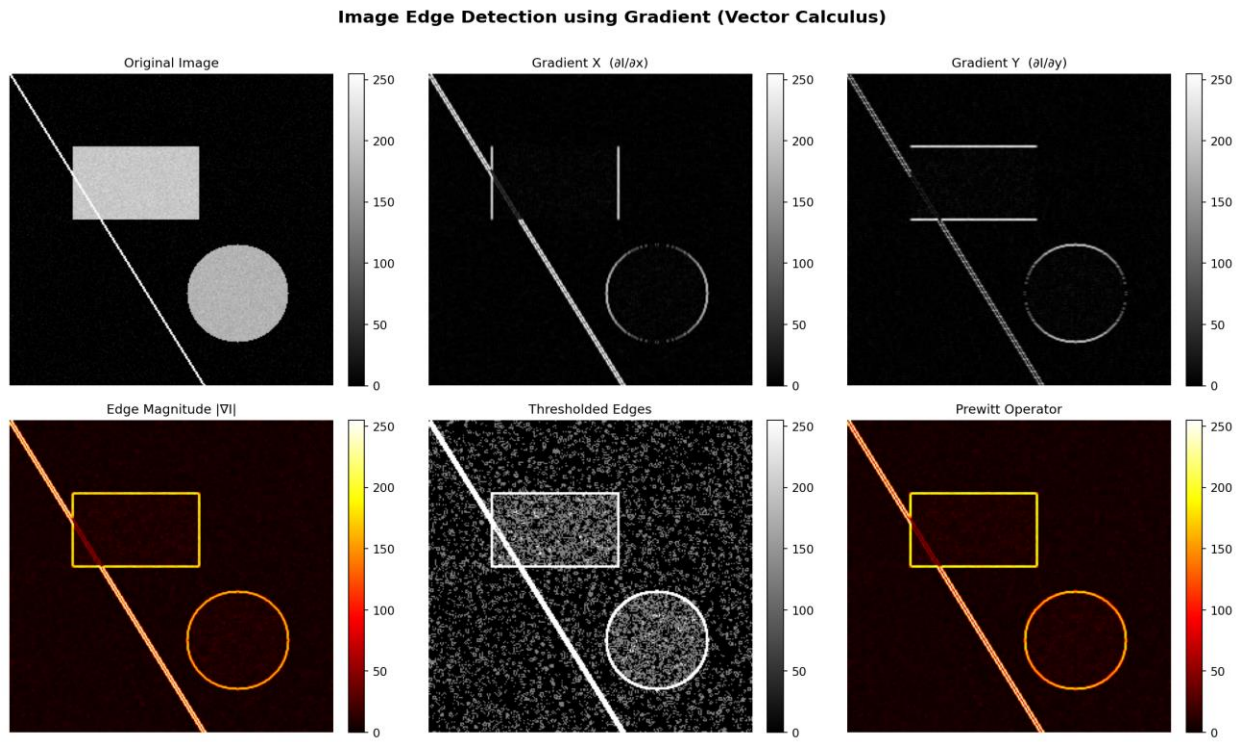

Figure 1: (top-left) Original Image, (top-middle) Gx = ∂I/∂x, (top-right) Gy = ∂I/∂y, (bottom-left) Edge Magnitude |∇I|, (bottom-middle) Thresholded Edges, (bottom-right) Prewitt Operator

5.2 Gradient Vector Field ∇I = (∂I/∂x) î + (∂I/∂y) ĵ

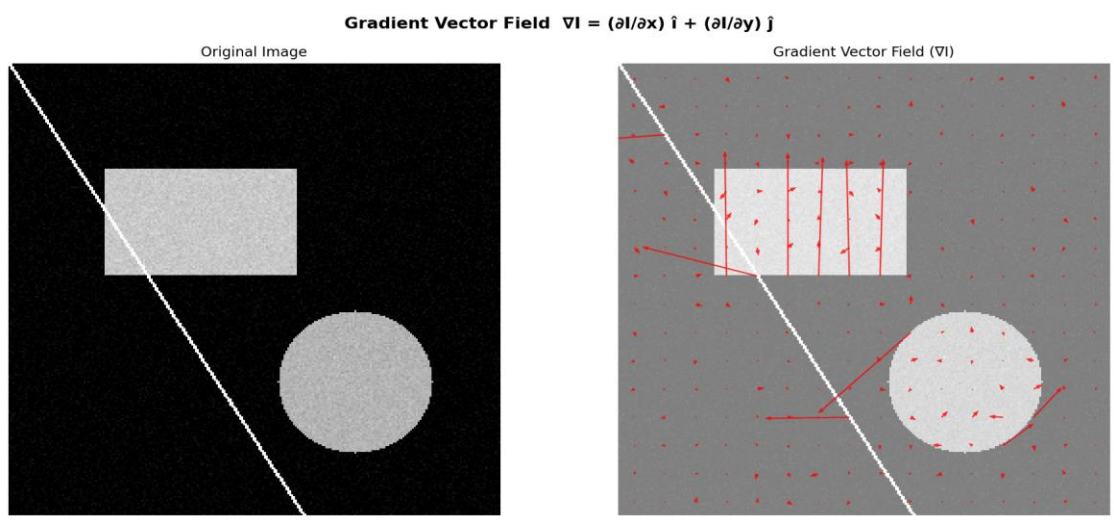

Figure 2: The red arrows show the gradient vectors at sampled points. Each arrow points in the direction of maximum intensity change, visually confirming edge locations.

5.3 Magnitude Heatmap & Edge Direction Map

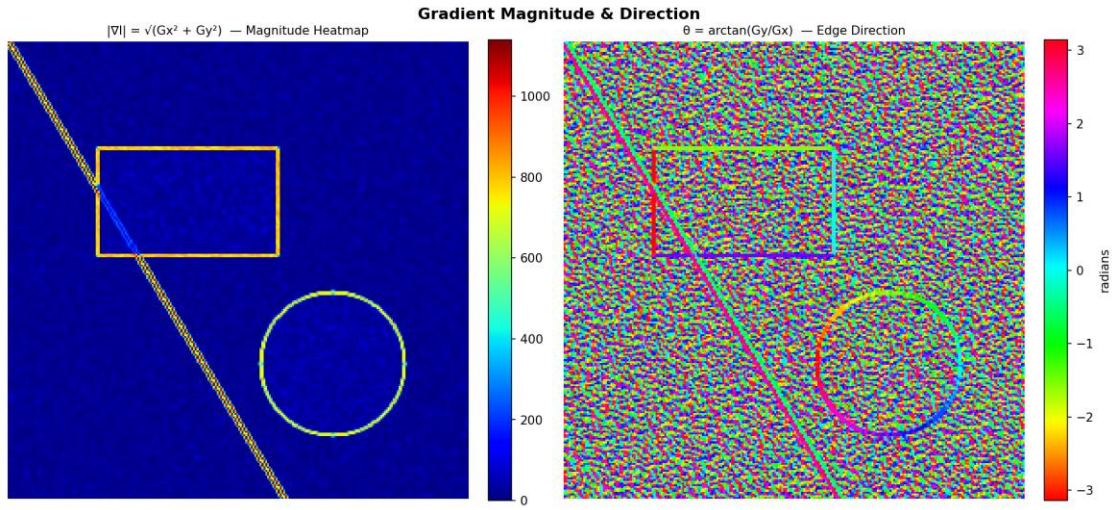

Figure 3: (left) Jet heatmap of gradient magnitude — brighter colours = stronger edges. (right) HSV colormap of gradient direction θ — different hues indicate edge orientations.

5.4 Threshold Sensitivity Analysis

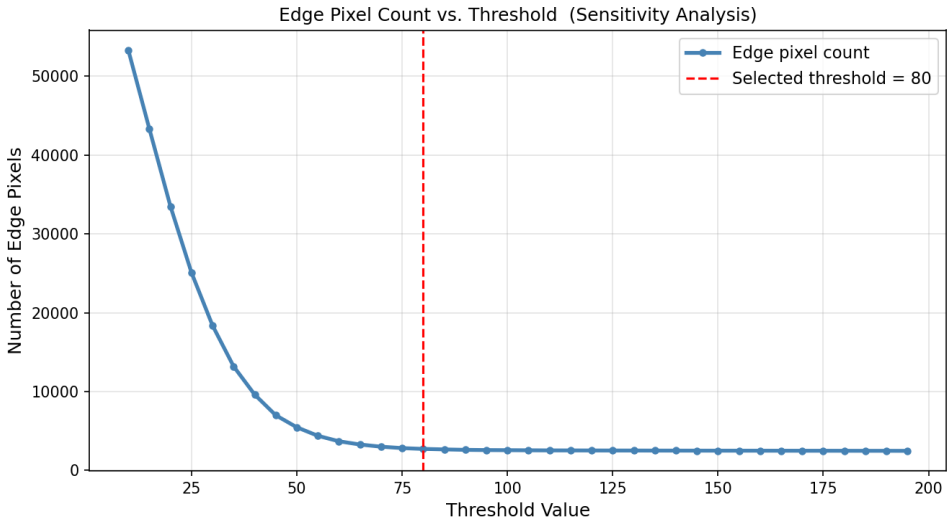

Figure 4: Edge pixel count vs. threshold value. The red dashed line marks the chosen threshold (80). Lower thresholds include more noise; higher thresholds miss weaker edges.

6. Testing and Results Analysis

The algorithm was applied to a 256×256 synthetic image containing a rectangle, a circle, and a diagonal line — all clearly identifiable geometric shapes.

Metric	Value	Description
Image Size	256 × 256 pixels	Synthetic test image dimensions
Max Gradient Mag	1139.48	Highest intensity change detected
Mean Gradient Mag	47.90	Average gradient across image
Strong Edges (>80)	2,717 pixels	Clearly defined edges
Weak Edges (30–80)	15,648 pixels	Potential/transitional edges
Direction Range	-π to +π radians	Full angular range captured

6.1 Observations

- The Sobel operator successfully detected all geometric boundaries in the image.
- The gradient magnitude heatmap (Figure 3) clearly highlights edges with high intensity values.
- The vector field (Figure 2) shows that gradient arrows point perpendicular to edge contours — consistent with the mathematical property that ∇I is perpendicular to level curves.
- The direction map (Figure 3 right) shows distinct colours for horizontal, vertical, and diagonal edges, confirming directional accuracy.
- Threshold analysis (Figure 4) shows that a threshold of 80 provides a good balance between edge completeness and noise suppression.
- The Prewitt operator produces similar results to Sobel but with slightly less noise suppression due to its simpler kernel weights.

7. How Vector Calculus Concepts Were Applied

Concept	Application in this Project
Scalar Field	Image I(x, y) modelled as a 2D scalar field mapping pixel coords to intensity
Gradient ∇f	∇I computed at every pixel — direction of maximum intensity change = edge direction
Partial Derivatives	∂I/∂x and ∂I/∂y computed via Sobel convolution — detect V and H edges respectively
Vector Field	∇I = (Gx)î + (Gy)ĵ forms a 2D vector field visualised as quiver plot
Magnitude |∇f|	|∇I| = √(Gx²+Gy²) gives edge strength — high magnitude = definite edge
Direction θ	θ = arctan(Gy/Gx) gives the edge orientation angle at each pixel

8. Conclusion

This project successfully demonstrated the application of vector calculus — specifically the concept of the gradient — to the real-world problem of image edge detection. By modelling an image as a 2D scalar field and computing its gradient vector field, we were able to precisely identify the location, strength, and orientation of edges.

The Sobel operator provides a practical discrete approximation to the continuous gradient, making it computationally efficient and widely used in computer vision systems. The double-threshold technique adds robustness by distinguishing strong definite edges from weaker candidate edges.

This approach forms the mathematical foundation for modern image processing pipelines, including the Canny edge detector, image segmentation, object recognition in AI systems, and feature extraction in convolutional neural networks (CNNs).